МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по выполнению индивидуального задания

по дисциплине «Технология разработки программного обеспечения»

по теме: «Создание подсистемы для работы с партнерами компании»

Работу выполнил

студент группы ИПо-42

В.С.Потеха

«16» марта 2026 г.

Пермь, 2026 г.

Оглавление

Введение ... 3
Глава 1. Анализ предметной области ... 5
1.1 Описание предметной области ... 5
1.2 Функциональные требования ... 6
Глава 2. Создание базы данных ... 8
2.1 Логическая модель данных ... 8
2.2 Физическая реализация базы данных ... 8
Глава 3. Создание программных библиотек ... 14
Глава 4. Тестирование библиотеки ... 21
4.1 Проверка работоспособности компонентов с помощью модульных тестов ... 21
4.2 Итоги проверки работоспособности ... 23
Глава 5. Создание клиентского приложения ... 25
5.1 Разработка пользовательского интерфейса ... 25
5.2 Реализация экранных форм ... 26
Заключение ... 32
Приложения ... 33

Введение

В настоящем отчете представлено описание процессов исследования предметной области предприятия-заказчика, проектирования структуры базы данных, разработки и тестирования приложения, обеспечивающего пользователю доступ к функциональным возможностям через графический интерфейс.

При создании программного продукта задействованы следующие средства разработки:

- система управления базами данных PostgreSQL;
- интегрированная среда разработки Visual Studio 2022;
- технология построения оконных приложений WPF.

Разработанное приложение включает библиотеку, реализующую четыре основных операции взаимодействия с базой данных: добавление новых записей, выборку (чтение) данных, изменение существующих записей и удаление объектов.

Для обеспечения взаимодействия клиентской части с базой данных применены дополнительные компоненты:

- Microsoft.EntityFrameworkCore — инструмент объектно-реляционного отображения, обеспечивающий работу с данными в объектной парадигме;
- Npgsql.EntityFrameWorkCore.PostgreSQL — драйвер, обеспечивающий совместимость Entity Framework с СУБД PostgreSQL;
- Microsoft.EntityFrameworkCore.Tools — набор служебных команд для импорта, экспорта и синхронизации данных между приложением и базой данных.

Задача, стоявшая в ходе выполнения работы, заключалась в создании приложения для управления базой данных, имеющей структуру «один ко многим». Построение пользовательского интерфейса выполнялось с

использованием технологии WPF, взаимодействие с данными обеспечивалось через Entity Framework Core.

Глава 1. Анализ предметной области

1.1 Описание предметной области

Перед началом проектирования системы был проведен анализ данных, с которыми предстоит работать пользователю, а также определен перечень информации, доступной для изменения. Детальное описание предметной области приведено в приложении 1.

В рамках выполнения работы требовалось создать программный модуль для ведения учета контрагентов предприятия, реализующий следующие возможности:

- отображение перечня партнеров;
- внесение новых записей и корректировка существующих сведений о партнерах;
- просмотр данных о продажах, осуществленных каждым партнером.

При формировании отчета по продажам предусмотрен вывод следующих сведений:

- название товара;
- объем реализованной продукции в количественном выражении;
- дата совершения операции.

При работе с карточкой партнера должны присутствовать поля для ввода и отображения:

- названия организации;
- категории контрагента;
- числового значения рейтинга;
- почтового адреса;
- фамилии, имени, отчества руководителя;
- контактного телефона;
- электронной почты.

Кроме того, техническим заданием предусмотрена возможность добавления, изменения и удаления не только записей о партнерах, но и связанных с ними объектов: категорий партнеров, номенклатуры продукции, а также данных о реализованных товарах.

На основе анализа требований была определена структура базы данных, включающая четыре основные таблицы:

- номенклатура продукции;
- категории партнеров;
- данные о партнерах;
- сведения о продажах, совершенных контрагентами.

1.2 Функциональные требования

Таблица продукции содержит только два поля: уникальный артикул (используется для идентификации товара) и наименование. Иные характеристики, такие как цена, изображение или описание, в рамках поставленной задачи не требовались и в разработке не учитывались.

Таблица категорий партнеров включает уникальное наименование категории без дополнительных атрибутов.

Таблица партнеров, помимо перечисленных выше реквизитов, содержит ссылку на категорию, к которой относится контрагент.

Таблица продаж фиксирует информацию о реализованной продукции: ссылку на товар, ссылку на партнера-продавца, количество проданных единиц и дату совершения операции.

Проведенный анализ позволил определить состав сущностей и их атрибутов, необходимых для последующего проектирования базы данных разрабатываемой системы.

- автоматический пересчет скидки партнера при любых изменениях данных о продажах;
- зависимость скидки от общего количества проданной продукции по следующей шкале:

1. до 10 000 единиц – скидка 0%;

2. от 10 001 до 50 000 единиц – скидка 5%;
3. от 50 001 до 300 000 единиц – скидка 10%;
4. более 300 000 единиц – скидка 15%.

Требования к пользовательскому интерфейсу:

- разделение главного окна на две области: список партнеров слева, история продаж справа;
- наличие главного меню для доступа ко всем функциям;
- наличие контекстного меню (правая кнопка мыши) для быстрого выполнения операций;
- наличие диалоговых окон для ввода и редактирования данных;
- информативные сообщения об ошибках с указанием причин и способов их устранения;
- подтверждение критических действий (удаление, выход из программы).

Глава 2. Создание базы данных

2.1 Логическая модель данных

Согласно техническому заданию, разрабатываемое приложение должно взаимодействовать с предварительно созданной базой данных и использовать для подключения определенную учетную запись. В связи с этим до начала непосредственной разработки программного кода выполняются подготовительные операции по развертыванию инфраструктуры хранения данных.

В ходе подготовки создается учетная запись (роль) с необходимыми правами доступа, выделяется табличное пространство для размещения данных, инициализируется сама база данных, а внутри нее формируется отдельная схема, владельцем которой выступает созданная ранее роль.

Связи между сущностями:

- «Тип партнера» и «Партнер» – связь один-ко-многим (один тип может быть у многих партнеров);
- «Партнер» и «Продажа» – связь один-ко-многим (один партнер может иметь много продаж);
- «Продукция» и «Продажа» – связь один-ко-многим (одна продукция может быть продана много раз).

Все связи обеспечивают ссылочную целостность: при удалении партнера каскадно удаляются все его продажи; при удалении типа партнера или продукции, имеющих связанные записи, операция запрещается.

2.2 Физическая реализация базы данных

Указанные операции выполняются с помощью команд, вводимых в терминальной среде PostgreSQL. Для отправки запросов используется встроенный консольный клиент psql. Полный перечень использованных команд с пояснениями приведен в листинге 2.1.

Листинг 2.1 – Набор команд для инициализации и управления базой данных

```
-- Создание роли
CREATE ROLE app2 WITH LOGIN PASSWORD '123456789';

-- Создание базы данных
CREATE DATABASE poteha
    WITH OWNER = app2
    ENCODING = 'UTF8'
    LC_COLLATE = 'ru_RU.UTF-8'
    LC_CTYPE = 'ru_RU.UTF-8'
    TEMPLATE = template0;

-- Подключение к базе данных
\c poteha app2;

-- Создание схемы
CREATE SCHEMA app AUTHORIZATION app2;
```

При создании учетной записи для подключения к базе данных использовались следующие параметры:

- имя роли: «app2»;
- установлены атрибуты «LOGIN» (разрешение на вход) и «PASSWORD» (назначение пароля);
- значение пароля для аутентификации: «123456789».

- После создания базы данных выполняется создание таблиц в соответствии с разработанной логической моделью. Скрипт создания таблиц представлен в листинге 2.2.

Листинг 2.2 – Создание таблиц базы данных

```sql
-- Создание таблицы партнеров
CREATE TABLE partners_poteha (
    id SERIAL PRIMARY KEY,
    type_id INTEGER NOT NULL REFERENCES partner_types_poteha(id) ON DELETE RESTRICT,
    name VARCHAR(200) NOT NULL,
    director_fullname VARCHAR(200),
    phone VARCHAR(20),
    email VARCHAR(100),
    rating INTEGER CHECK (rating >= 0),
    discount INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Создание таблицы продукции
CREATE TABLE products_poteha (
    id SERIAL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    article VARCHAR(50) UNIQUE,
    price DECIMAL(10, 2) CHECK (price > 0)
);

-- Создание таблицы продаж
CREATE TABLE sales_poteha (
    id SERIAL PRIMARY KEY,
    partner_id INTEGER NOT NULL REFERENCES partners_poteha(id) ON DELETE CASCADE,
    product_id INTEGER NOT NULL REFERENCES products_poteha(id) ON DELETE RESTRICT,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    sale_date DATE NOT NULL DEFAULT CURRENT_DATE,
    total_amount DECIMAL(10, 2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Для обеспечения автоматического пересчета скидки при изменении данных о продажах созданы триггеры и функции, представленные в листинге 2.3.

Листинг 2.3 – Триггеры для автоматического расчета скидки

```
-- Функция для расчета скидки
CREATE OR REPLACE FUNCTION calculate_partner_discount_poteha(p_partner_id INTEGER)
RETURNS INTEGER AS $$
DECLARE
    total_quantity INTEGER;
    new_discount INTEGER;
BEGIN
    -- Считаем общее количество продаж для партнера
    SELECT COALESCE(SUM(quantity), 0) INTO total_quantity
    FROM sales_poteha
    WHERE partner_id = p_partner_id;

    -- Определяем скидку по шкале
    IF total_quantity < 10000 THEN
        new_discount := 0;
    ELSIF total_quantity < 50000 THEN
        new_discount := 5;
    ELSIF total_quantity < 300000 THEN
        new_discount := 10;
    ELSE
        new_discount := 15;
    END IF;

    RETURN new_discount;
END;
$$ LANGUAGE plpgsql;

-- Функция для обновления скидки
CREATE OR REPLACE FUNCTION update_partner_discount_poteha(p_partner_id INTEGER)
RETURNS VOID AS $$
BEGIN
    UPDATE partners_poteha
    SET discount = calculate_partner_discount_poteha(p_partner_id)
    WHERE id = p_partner_id;
END;
$$ LANGUAGE plpgsql;

-- Триггерная функция
CREATE OR REPLACE FUNCTION trigger_update_partner_discount_poteha()
RETURNS TRIGGER AS $$
DECLARE
    affected_partner_id INTEGER;
BEGIN
    IF TG_OP = 'DELETE' THEN
        affected_partner_id := OLD.partner_id;
    ELSE
        affected_partner_id := NEW.partner_id;
    END IF;

    PERFORM update_partner_discount_poteha(affected_partner_id);
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

После создания таблиц выполняется наполнение базы данных тестовыми значениями (листинг 2.4).

Листинг 2.4 – Наполнение базы тестовыми данными

```
-- Вставка начальных данных
INSERT INTO partner_types_poteha (name) VALUES
    ('ООО'),
    ('ЗАО'),
    ('ИП'),
    ('АО'),
    ('ПАО');

-- Вставляем продукты (универсальные названия)
INSERT INTO products_poteha (name, article, price) VALUES
    ('Продукт 1', 'ART001', 1000.00),
    ('Продукт 2', 'ART002', 2500.00),
    ('Продукт 3', 'ART003', 500.00),
    ('Продукт 4', 'ART004', 3000.00),
    ('Продукт 5', 'ART005', 1500.00);

-- Добавим тестовых партнеров
INSERT INTO partners_poteha (type_id, name, director_fullname, phone, email, rating)
VALUES
    (1, 'Ромашка', 'Иванов Иван Иванович', '+7(999)123-45-67', 'info@romashka.ru', 5),
    (2, 'ТехноПром', 'Петров Петр Петрович', '+7(999)765-43-21', 'info@techno.ru', 4),
    (3, 'Сидоров и К', 'Сидоров Сидор Сидорович', '+7(999)555-55-55', 'sidorov@mail.ru', 3);
```

– После завершения подготовительных операций и создания необходимых объектов базы данных, ее наполнение таблицами выполнялось с использованием ранее созданной роли. Формирование структуры таблиц осуществлялось в точном соответствии со схемой, представленной на рисунке 2.1.

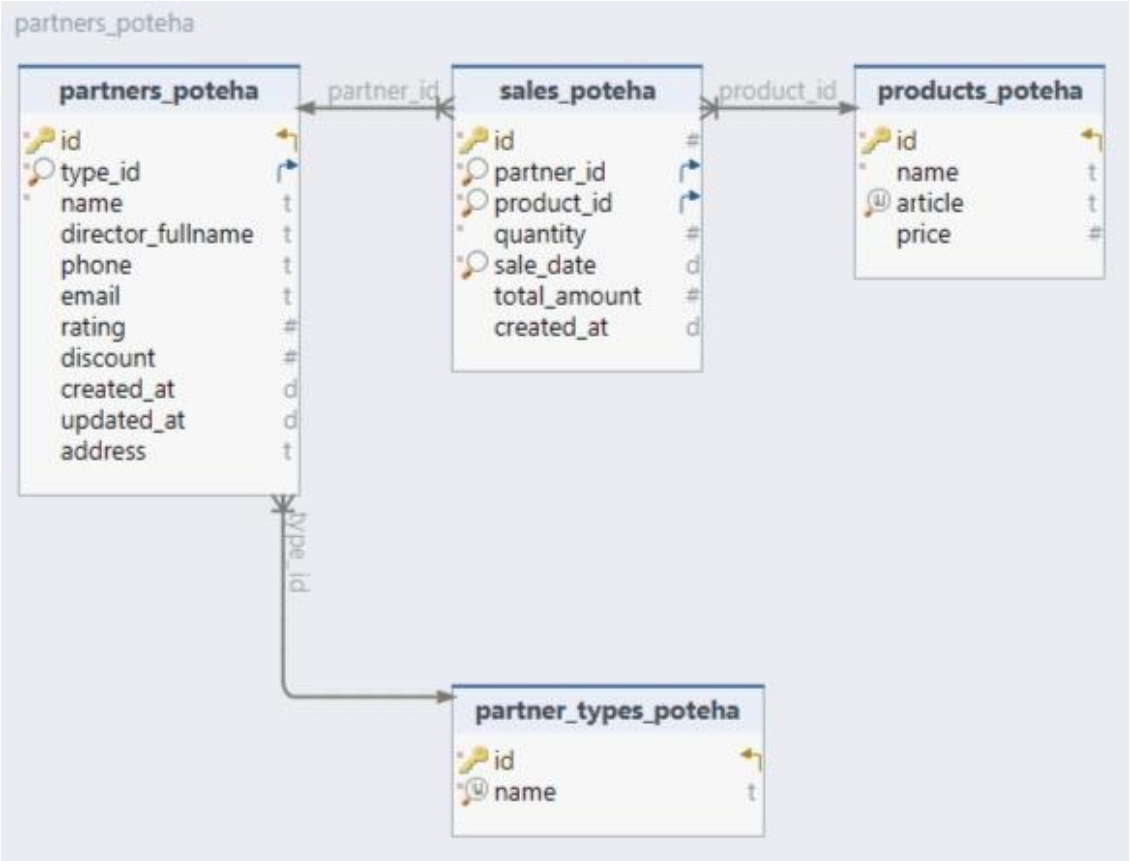

Рисунок 2.1 – Схема базы данных

При проектировании таблиц использовались следующие типы данных:

- SERIAL — целочисленный тип с автоматическим увеличением значения, применялся для создания первичных ключей;
- INT — стандартный целочисленный тип, использовался для хранения числовых значений и внешних ключей;
- VARCHAR(n) — строковый тип с ограничением максимальной длины;
- DATE — тип для хранения календарных дат.

Для внешних ключей были настроены каскадные операции обновления и удаления, обеспечивающие автоматическую синхронизацию связанных записей при изменении или удалении родительских данных.

К полям, содержащим рейтинг партнера и количество реализованной продукции, добавлено ограничение, проверяющее, что вводимое значение не может быть отрицательным (условие «больше или равно нулю»).

В качестве наполнителя для тестирования структуры в таблицы были добавлены начальные записи: перечень категорий партнеров и список товаров.

После завершения формирования базы данных и ее наполнения тестовыми данными был выполнен переход к следующему этапу — созданию библиотек, обеспечивающих программное взаимодействие с базой данных.

Глава 3. Создание программных библиотек

В ходе выполнения работы предусмотрена разработка трех программных компонентов, обеспечивающих взаимодействие с базой данных:

- библиотека, содержащая модели, соответствующие структуре таблиц;
- библиотека с контекстом базы данных, управляющим подключениями и запросами;
- библиотека, реализующая основные операции обработки данных: получение, добавление, изменение и удаление записей.

Для автоматического формирования моделей по существующей структуре базы данных задействован инструментарий, входящий в состав пакета «Microsoft.EntityFrameworkCore.Tools». С помощью команды «Scaffold-DbContext» (приведенной в листинге 3.1) была выполнена генерация классов-сущностей на языке C#, соответствующих таблицам базы данных, а также создан контекст, обеспечивающий взаимодействие с этими таблицами. Полученный контекст впоследствии был вынесен в отдельную библиотеку для соблюдения принципов модульности.

Листинг 3.1 – Команда для создания классов-сущностей по структуре базы данных

```
Ссылок: 0
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.UseNpgsql("Host=localhost;Database=poteha;Username=app2;Password=123456789;Encoding=UTF8;Search Path=app");
    optionsBuilder.EnableSensitiveDataLogging();
    optionsBuilder.EnableDetailedErrors();
}
```

При вызове команды генерации моделей необходимо задать следующие параметры:

- строка соединения с базой данных, содержащая адрес сервера, порт, имя базы, учетные данные пользователя;
- идентификатор провайдера, соответствующий используемой СУБД;

– пути к проектам, в которых установлен требуемый пакет и куда будут помещены создаваемые классы;

– имя схемы базы данных, из которой выполняется импорт;

– наименование класса контекста, управляющего доступом к данным.

После успешного выполнения команды и распределения сгенерированных файлов по соответствующим проектам сформировалась структура каталогов и файлов, отображенная на рисунке 3.1.

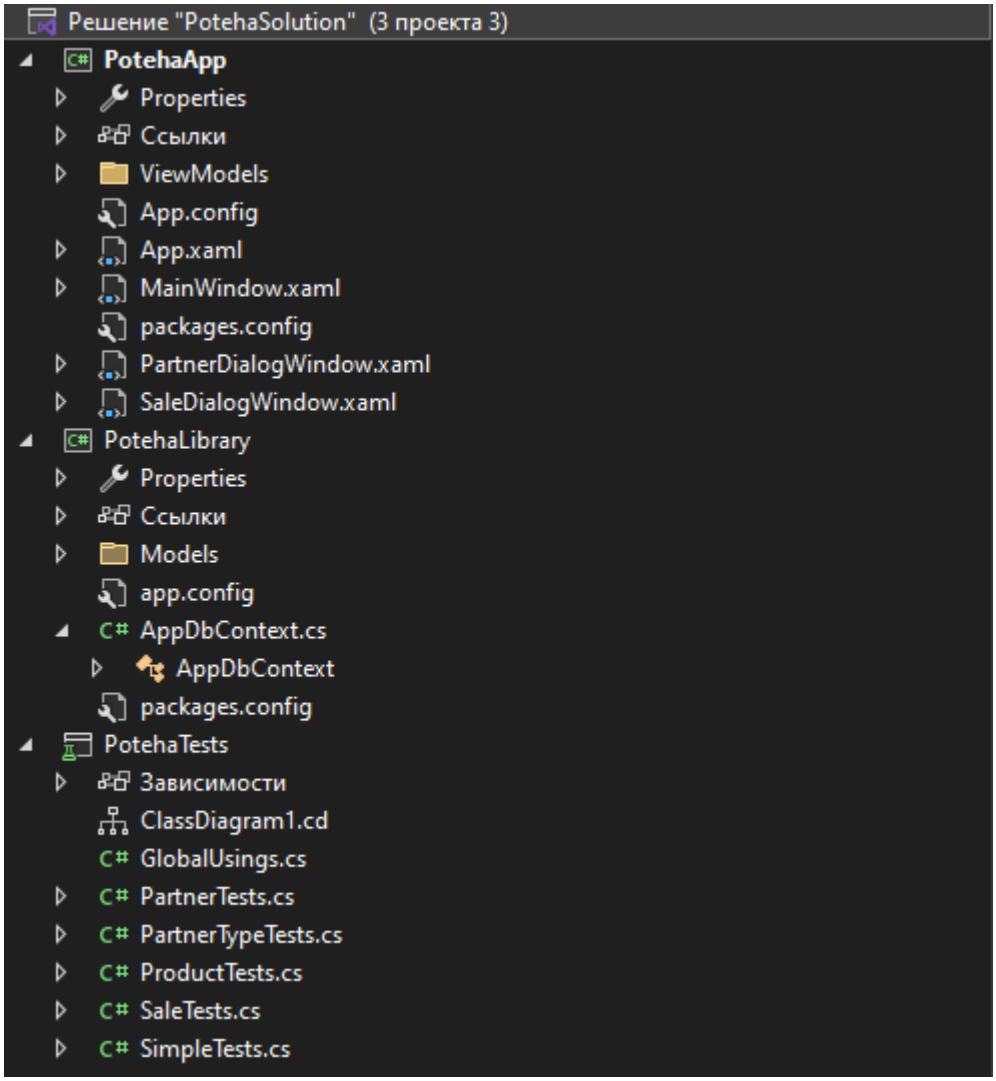

Рисунок 3.1 – Организация файлов и каталогов в проектах с моделями данных и контекстом

Визуальное представление структуры сгенерированных классов-сущностей и контекста базы данных приведено на рисунках 3.2 и 3.3

соответственно. На данных иллюстрациях можно ознакомиться с составом свойств каждой модели и методами, доступными в контексте для взаимодействия с таблицами.

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Storage.ValueConversion;
using PotehaLibrary.Models;
using System;

namespace PotehaLibrary
{
    Ссылок: 6
    public class AppDbContext : DbContext
    {
        Ссылок: 3
        public DbSet<PartnerType> PartnerTypes { get; set; }
        Ссылок: 5
        public DbSet<Partner> Partners { get; set; }
        Ссылок: 2
        public DbSet<Product> Products { get; set; }
        Ссылок: 10
        public DbSet<Sale> Sales { get; set; }

        Ссылок: 0
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseNpgsql("Host=localhost;Database=poteha;Username=app2;Password=123456789;Encoding=UTF8;Search Path=app");
            optionsBuilder.EnableSensitiveDataLogging();
            optionsBuilder.EnableDetailedErrors();
        }

        Ссылок: 0
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.HasDefaultSchema("app");

            modelBuilder.Entity<PartnerType>().ToTable("partner_types_poteha");
            modelBuilder.Entity<Partner>().ToTable("partners_poteha");
            modelBuilder.Entity<Product>().ToTable("products_poteha");
            modelBuilder.Entity<Sale>().ToTable("sales_poteha");

            modelBuilder.Entity<PartnerType>(entity =>
            {
                entity.HasKey(e => e.Id);
                entity.Property(e => e.Id).HasColumnName("id");
                entity.Property(e => e.Name).HasColumnName("name").IsRequired().HasMaxLength(100);
            });

            modelBuilder.Entity<Partner>(entity =>
            {
                entity.HasKey(e => e.Id);
                entity.Property(e => e.Id).HasColumnName("id");
                entity.Property(e => e.TypeId).HasColumnName("type_id");
                entity.Property(e => e.Name).HasColumnName("name").IsRequired().HasMaxLength(200);
                entity.Property(e => e.DirectorFullname).HasColumnName("director_fullname").HasMaxLength(200);
                entity.Property(e => e.Phone).HasColumnName("phone").HasMaxLength(20);
                entity.Property(e => e.Email).HasColumnName("email").HasMaxLength(100);
                entity.Property(e => e.Rating).HasColumnName("rating");
                entity.Property(e => e.Discount).HasColumnName("discount");
                entity.Property(e => e.CreatedAt).HasColumnName("created_at");
                entity.Property(e => e.UpdatedAt).HasColumnName("updated_at");

                entity.HasOne(e => e.PartnerType)
                    .WithMany()
                    .HasForeignKey(e => e.TypeId)
                    .OnDelete(DeleteBehavior.Restrict);
```

Рисунок 3.2 – Частичное содержание контекста базы данных

```
{
    [Table("partners_poteha", Schema = "app")]
    Ссылок: 17
    public class Partner
    {
        [Key]
        [Column("id")]
        Ссылок: 6
        public int Id { get; set; }

        [Column("type_id")]
        Ссылок: 5
        public int TypeId { get; set; }

        [Required]
        [Column("name")]
        [MaxLength(200)]
        Ссылок: 7
        public string Name { get; set; }

        [Column("director_fullname")]
        [MaxLength(200)]
        Ссылок: 7
        public string DirectorFullname { get; set; }

        [Column("phone")]
        [MaxLength(20)]
        Ссылок: 7
        public string Phone { get; set; }

        [Column("email")]
        [MaxLength(100)]
        Ссылок: 4
        public string Email { get; set; }

        // ДОБАВЛЯЕМ ПОЛЕ АДРЕС
        [Column("address")]
        Ссылок: 3
        public string Address { get; set; }

        [Column("rating")]
        Ссылок: 7
        public int? Rating { get; set; }

        [Column("discount")]
        Ссылок: 6
        public int Discount { get; set; }

        [Column("created_at")]
        Ссылок: 5
        public DateTime CreatedAt { get; set; }

        [Column("updated_at")]
        Ссылок: 5
        public DateTime UpdatedAt { get; set; }

        Ссылок: 3
        public virtual PartnerType PartnerType { get; set; }
        Ссылок: 3
        public virtual ICollection<Sale> Sales { get; set; }
    }
}
```

Рисунок 3.3 – Содержание модели партнера

Из представленных иллюстраций видно, что автоматически созданные классы-сущности содержат лишь навигационные свойства для доступа к связанным объектам. Вся информация о подключении к базе данных (адрес сервера, учетные данные), а также детали таблиц и столбцов, ограничения и ключи вынесены в методы контекста.

Для настройки взаимодействия с базой данных применяются компоненты Entity Framework Core:

- DbContext — базовый класс, обеспечивающий соединение с базой и управление запросами;
- DbSet<TEntity> — коллекция, представляющая содержимое конкретной таблицы и позволяющая выполнять операции с соответствующими сущностями;

– DbContextOptionsBuilder — инструмент для задания параметров подключения;

– ModelBuilder — набор методов, используемых для конфигурирования отображения классов на таблицы базы данных.

Основным классом для работы с данными выступает AppDbContext, унаследованный от DbContext. Каждая таблица представлена в нем свойством типа DbSet, параметризованным соответствующей моделью.

На следующем этапе разрабатываются сервисные классы, инкапсулирующие логику взаимодействия с каждой сущностью. Для каждого типа данных создается отдельный сервис, реализующий базовые операции: получение списка записей, добавление, изменение и удаление. Для каждого сервиса также определяется соответствующий интерфейс, обеспечивающий слабую связанность компонентов. Примерная структура интерфейса и реализующего его сервиса приведена на рисунках 3.4 и 3.5.

```
using PotehaLibrary.Models;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace PotehaApp.ViewModels
{
    Ссылок: 9
    public class PartnerViewModel : INotifyPropertyChanged
    {
        private Partner _partner;

        Ссылок: 1
        public PartnerViewModel(Partner partner)
        {
            _partner = partner;
        }

        Ссылок: 10
        public int Id => _partner.Id;
        Ссылок: 5
        public string DisplayName => $"{_partner.PartnerType?.Name} / {_partner.Name}";
        Ссылок: 4
        public string Director => _partner.DirectorFullname ?? "Не указан";
        Ссылок: 4
        public string Phone => _partner.Phone ?? "Не указан";
        Ссылок: 7
        public int? Rating => _partner.Rating;
        Ссылок: 4
        public int Discount => _partner.Discount;

        Ссылок: 0
        public string Address => _partner.Address ?? "Не указан";

        Ссылок: 2
        public Partner Partner => _partner;

        public event PropertyChangedEventHandler PropertyChanged;
        Ссылок: 0
        protected void OnPropertyChanged([CallerMemberName] string name = null)
        {
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
        }
    }
}
```

Рисунок 3.4 – Структура интерфейса для взаимодействия с данными партнеров

```
using PotehaLibrary.Models;
using System;

namespace PotehaApp.ViewModels
{

    public class SaleViewModel
    {
        private Sale _sale;


        public SaleViewModel(Sale sale)
        {
            _sale = sale;
        }


        public int Id => _sale.Id;

        public string ProductName => _sale.Product?.Name ?? "Неизвестно";

        public int Quantity => _sale.Quantity;

        public DateTime SaleDate => _sale.SaleDate;

        public decimal TotalAmount => _sale.TotalAmount;
    }
}
```

Рисунок 3.5 – Пример части программного сервиса для работы с партнерами

При реализации операций добавления, изменения и удаления записей в сервисные методы передаются следующие параметры:

- для создания новой записи — объект, содержащий значения всех полей, кроме первичного ключа (генерируется автоматически);
- для обновления существующей записи — идентификатор изменяемого объекта и набор новых значений для изменяемых полей;
- для удаления записи — идентификатор объекта, подлежащего удалению.

Операции получения данных, как правило, не требуют передачи параметров, за исключением случая с историей продаж: для отображения списка реализованной продукции конкретного партнера методу необходимо передать его идентификатор.

Перед выполнением любой операции с данными в сервисах осуществляется проверка корректности переданных значений на соответствие требованиям, заданным структурой базы данных. При обнаружении недопустимых данных инициируется исключение, которое прерывает выполнение операции и передает управление вызывающему коду.

Сформированное сообщение об ошибке содержит информацию о причине сбоя, что позволяет впоследствии уведомить пользователя. В большинстве случаев проверка реализуется с помощью условных конструкций. Для валидации данных партнера дополнительно применяются регулярные выражения, контролирующие формат телефонного номера и адреса электронной почты.

В целях упрощения сигнатур методов и повышения удобства использования в отдельных случаях применяются составные объекты-посредники (DTO), объединяющие несколько связанных параметров. Такой подход позволяет передавать все необходимые данные одной сущностью, сокращая количество аргументов методов.

Итоговая структура разработанных библиотек и взаимосвязи между их компонентами отражены на диаграмме классов, представленной на рисунке 3.6.

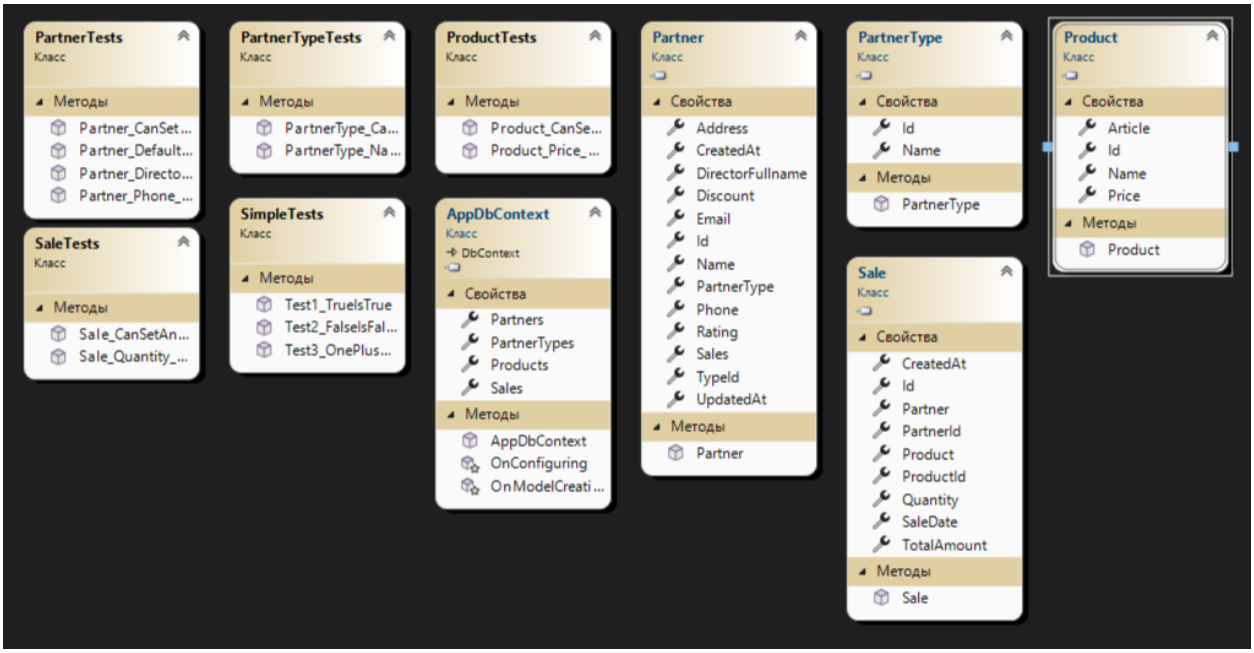

Рисунок 3.6 – Диаграмма классов

Глава 4. Тестирование библиотеки

4.1 Проверка работоспособности компонентов с помощью модульных тестов

Для подтверждения корректности функционирования основных модулей разработанной системы был подготовлен комплект автоматических тестов. В качестве инструмента тестирования выбран фреймворк xUnit. Проверке подлежали модели данных, классы-посредники ViewModel и алгоритм вычисления персональной скидки партнера.

Тестовый проект организован как отдельный компонент решения, не зависящий от остальных частей приложения. Все проверки выполняются изолированно, без обращения к реальной базе данных — тестируемые объекты создаются и используются непосредственно в оперативной памяти, что обеспечивает высокую скорость выполнения и воспроизводимость результатов.

В ходе тестирования моделей данных (результаты представлены на рисунке 4.1) проверялась способность объектов корректно сохранять и возвращать значения, присвоенные их свойствам. Для каждой модели выполнялась последовательность действий: создание экземпляра, заполнение полей тестовыми данными и последующее считывание значений с проверкой соответствия ожидаемым.

```
using Xunit;
using PotehaLibrary.Models;
using System.Collections.Generic;

namespace PotehaTests
{
    Ссылок: 0
    public class PartnerTests
    {
        [Fact]
        Ссылок: 0
        public void Partner_DefaultDiscount_IsZero()
        {
            // Arrange
            var partner = new Partner();

            // Act
            var discount = partner.Discount;

            // Assert
            Assert.Equal(0, discount);
        }

        [Fact]
        Ссылок: 0
        public void Partner_CanSetAndGetProperties()
        {
            // Arrange
            var partner = new Partner();

            // Act
            partner.Id = 10;
            partner.Name = "Тестовый партнер";
            partner.Rating = 5;
            partner.Discount = 10;

            // Assert
            Assert.Equal(10, partner.Id);
            Assert.Equal("Тестовый партнер", partner.Name);
            Assert.Equal(5, partner.Rating);
            Assert.Equal(10, partner.Discount);
        }

        [Fact]
        Ссылок: 0
        public void Partner_Director_CanBeNull()
        {
            // Arrange
            var partner = new Partner();

            // Act
            partner.DirectorFullname = null;

            // Assert
            Assert.Null(partner.DirectorFullname);
        }

        [Fact]
        Ссылок: 0
        public void Partner_Phone_CanBeNull()
        {
            // Arrange
            var partner = new Partner();

            // Act
            partner.Phone = null;

            // Assert
            Assert.Null(partner.Phone);
        }
    }
}
```

Рисунок 4.1 – Тест модели партнера

В рамках тестирования, результаты которого отражены на рисунке 4.2, проверялась правильность преобразования данных для их последующего отображения в пользовательском интерфейсе, а также корректность обработки ситуаций, когда значения свойств моделей не заданы (равны null). Особое внимание уделялось поведению компонентов при отсутствии данных и корректности формирования строковых представлений для вывода на экран.

```
using Xunit;
using PotehaLibrary.Models;
using System;

namespace PotehaTests
{
    public class SaleTests
    {
        [Fact]
        public void Sale_CanSetAndGetProperties()
        {
            // Arrange
            var sale = new Sale();
            var testDate = DateTime.Now;

            // Act
            sale.Id = 15;
            sale.PartnerId = 3;
            sale.ProductId = 7;
            sale.Quantity = 25;
            sale.TotalAmount = 5000m;
            sale.SaleDate = testDate;

            // Assert
            Assert.Equal(15, sale.Id);
            Assert.Equal(3, sale.PartnerId);
            Assert.Equal(7, sale.ProductId);
            Assert.Equal(25, sale.Quantity);
            Assert.Equal(5000m, sale.TotalAmount);
            Assert.Equal(testDate, sale.SaleDate);
        }

        [Fact]
        public void Sale_Quantity_Positive()
        {
            // Arrange
            var sale = new Sale();

            // Act
            sale.Quantity = 1;

            // Assert
            Assert.True(sale.Quantity > 0);
        }
    }
}
```

Рисунок 4.2 – Тест Sale

4.2 Итоги проверки работоспособности

Разработанные сценарии автоматической проверки были полностью пройдены без ошибок. Все компоненты продемонстрировали ожидаемое поведение в соответствии с требованиями. Итоговые результаты выполнения тестов представлены на рисунке 4.3.

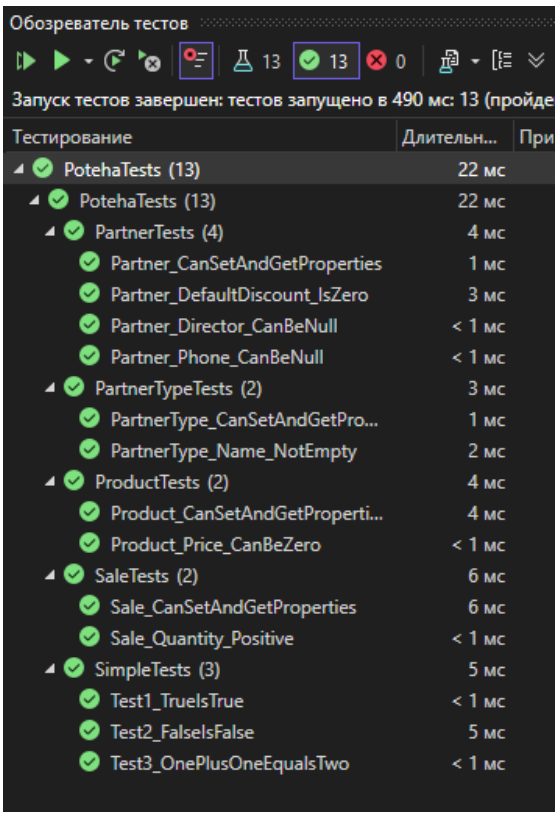

Рисунок 4.3 – Результаты выполнения тестов

Успешное прохождение всех тестов подтверждает корректность работы основных компонентов системы и их готовность к интеграции в клиентское приложение.

Глава 5. Создание клиентского приложения

5.1 Разработка пользовательского интерфейса

Для создания графической оболочки приложения использована технология Windows Presentation Foundation (WPF), предоставляющая широкие возможности по построению современных оконных интерфейсов.

В процессе проектирования были определены три основные экранные формы, обеспечивающие взаимодействие пользователя с системой:

- Основное окно — центральный элемент приложения, объединяющий перечень контрагентов и сведения о продажах выбранного партнера;
- Окно редактирования данных партнера — форма, предназначенная для ввода новой информации и изменения существующих записей о партнерах;
- Окно регистрации продаж — диалог для добавления и корректировки данных о реализованной продукции.

Эскиз главного окна приведен на рисунке 5.1. Рабочая область разделена на две функциональные зоны: в левой части экрана размещен список всех партнеров, правая часть отведена под отображение истории продаж, относящихся к выбранному в списке контрагенту. В верхней части окна расположено главное меню, через которое пользователь получает доступ ко всем возможностям приложения.

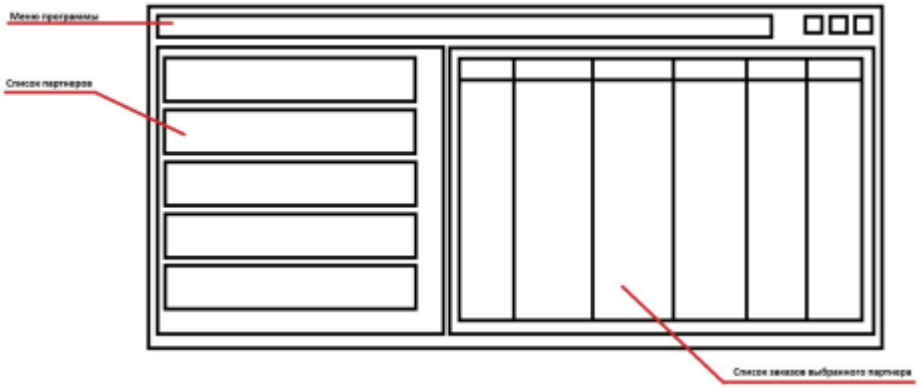

Рисунок 5.1 – Прототип главного окна

Для повышения удобства взаимодействия с системой были внедрены следующие элементы управления:

- контекстное меню — активируется нажатием правой кнопки мыши и предоставляет набор команд, доступных для текущего выбранного элемента;
- сочетания клавиш — назначены для часто используемых операций: клавиша F5 обновляет содержимое списков, F1 открывает справочную информацию, F2 позволяет редактировать выделенную запись, Delete удаляет выбранный объект;
- визуальные индикаторы — выделение активных элементов цветом, а также понятные уведомления при возникновении ошибок помогают пользователю ориентироваться в приложении.

5.2 Реализация экранных форм

Главное окно программы (изображено на рисунке 5.2) построено с применением стандартных элементов управления Menu, ListBox и DataGrid. Список контрагентов выводится с помощью ListBox, для которого создан индивидуальный шаблон отображения данных. В шаблоне предусмотрено отображение всей значимой информации о партнере, включая размер действующей скидки. История продаж представлена в виде таблицы DataGrid, содержащей фиксированный набор столбцов для вывода необходимых сведений.

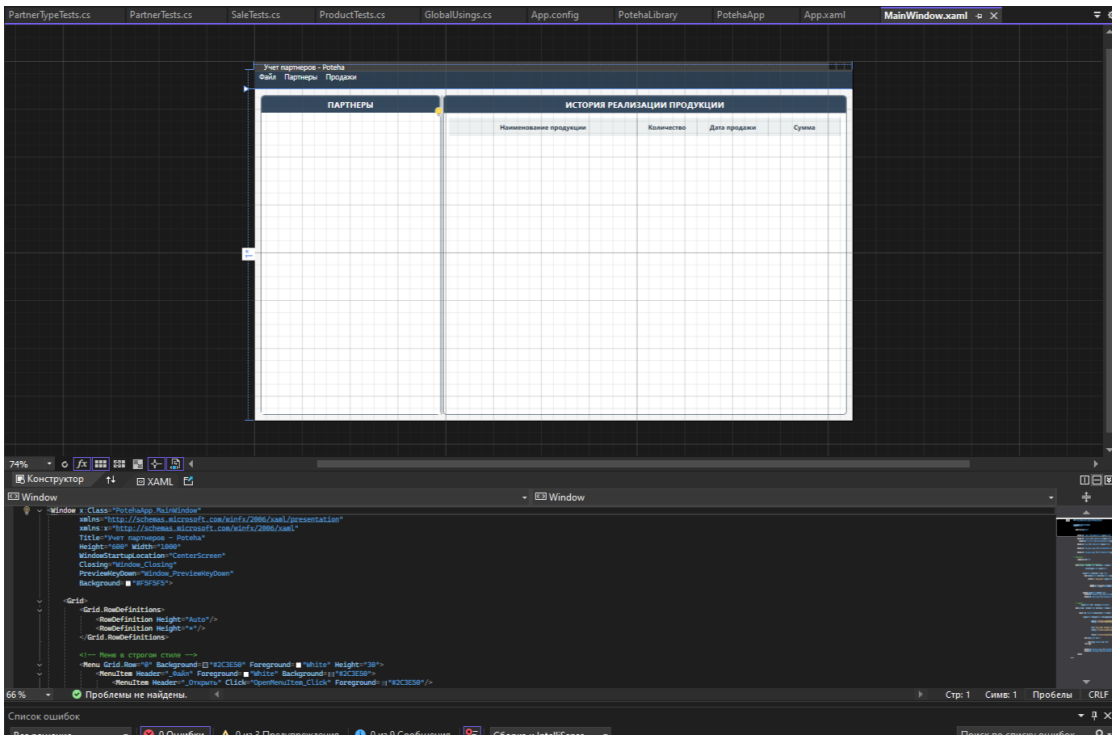

Рисунок 5.2 – Фрагмент разметки главного окна

Форма для работы с данными партнера, представленная на рисунке 5.3, включает набор полей, соответствующих всем обязательным характеристикам контрагента. Поле, предназначенное для указания рейтинга, настроено на прием только числовых значений, что исключает возможность ошибочного ввода букв или иных символов. Поле, отображающее размер скидки, заблокировано для редактирования и служит исключительно для информирования пользователя о текущем значении, рассчитанном автоматически.

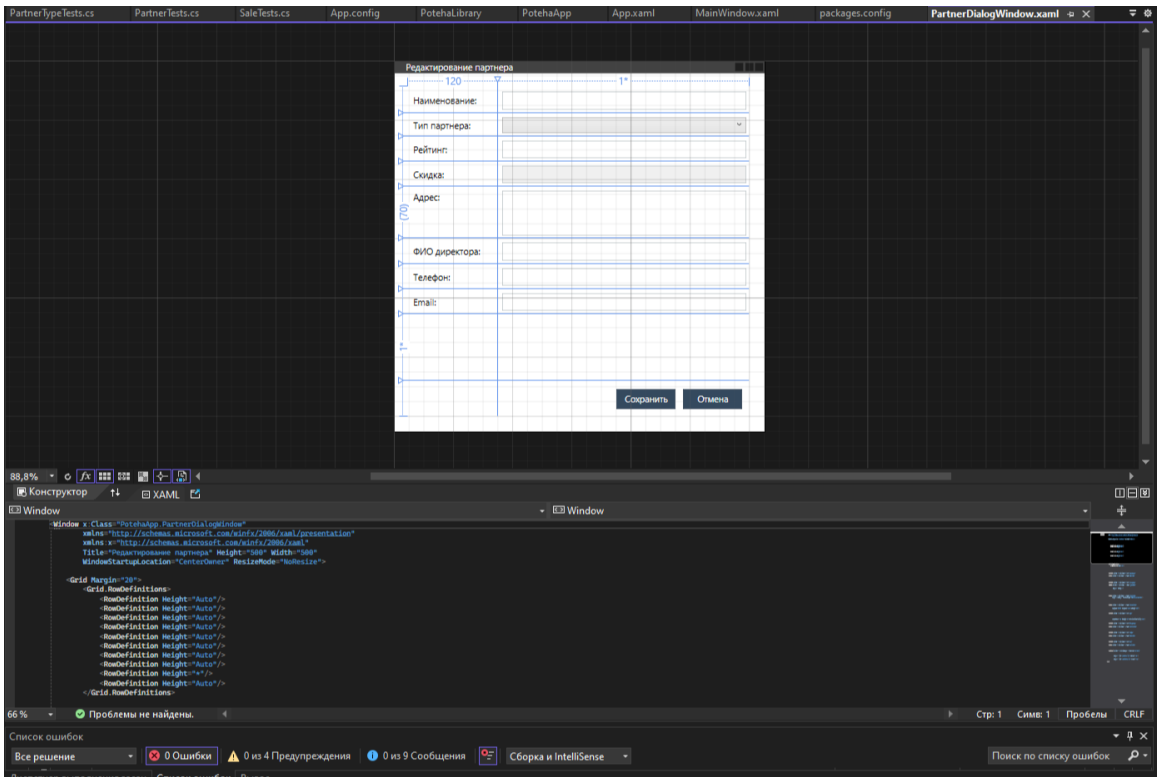

Рисунок 5.3 – Диалоговое окно партнера

Окно для оформления продажи (рисунок 5.4) содержит выпадающий список для выбора товара, поле ввода количества с контролем допустимых значений, элемент выбора даты и автоматически рассчитываемую сумму.

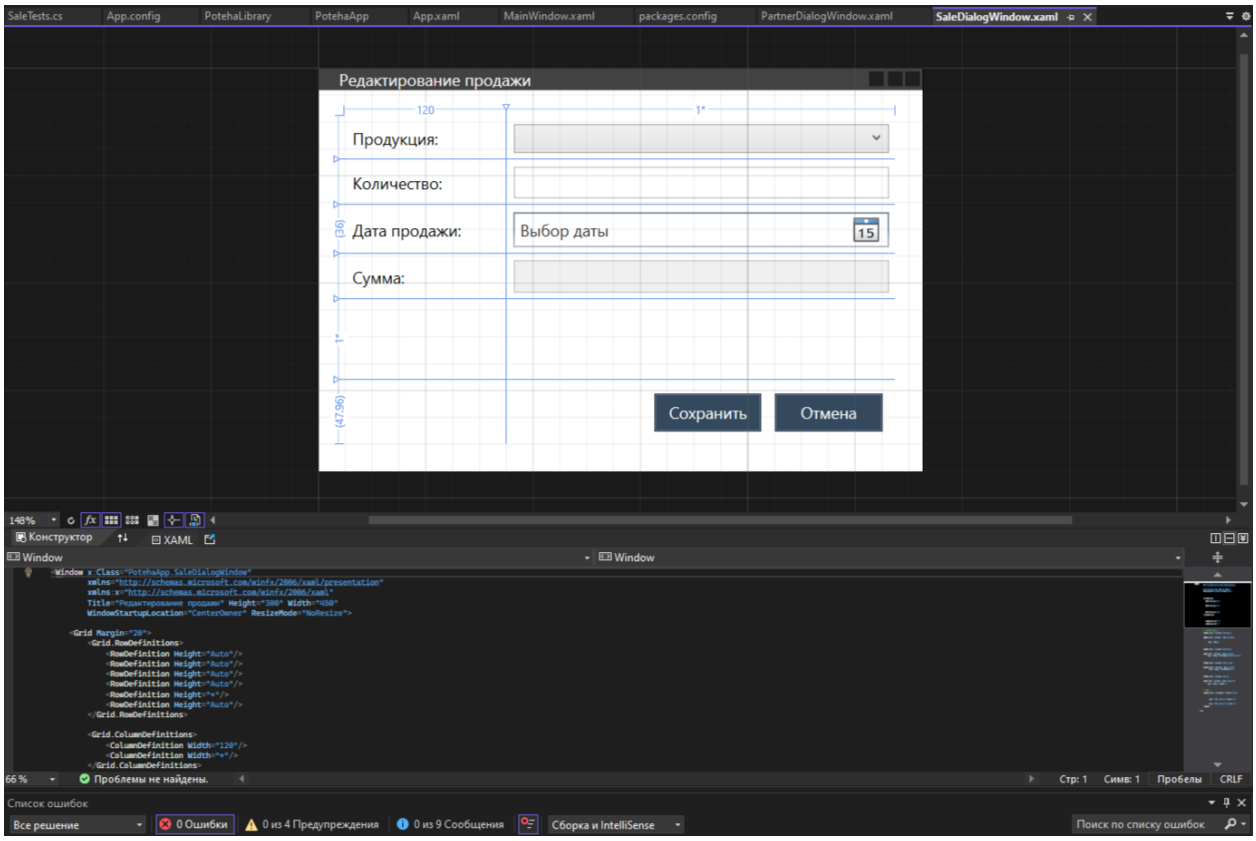

Рисунок 5.4 – Диалоговое окно продажи

В целях поддержания одинакового оформления всех экранных форм в приложении применяются общие стилевые решения, описанные в разделе ресурсов. Фрагмент соответствующей конфигурации представлен на рисунке 5.5.

```
<Application x:Class="PotehaApp.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             StartupUri="MainWindow.xaml">
    <Application.Resources>
        <!-- Стили для кнопок -->
        <Style TargetType="Button">
            <Setter Property="Padding" Value="10,5"/>
            <Setter Property="FontSize" Value="12"/>
            <Setter Property="Background" Value="#34495E"/>
            <Setter Property="Foreground" Value="White"/>
            <Setter Property="BorderBrush" Value="#2C3E50"/>
            <Setter Property="BorderThickness" Value="1"/>
            <Setter Property="Cursor" Value="Hand"/>
            <Style.Triggers>
                <Trigger Property="IsMouseOver" Value="True">
                    <Setter Property="Background" Value="#2C3E50"/>
                </Trigger>
                <Trigger Property="IsPressed" Value="True">
                    <Setter Property="Background" Value="#1E2B38"/>
                </Trigger>
            </Style.Triggers>
        </Style>

        <!-- Стили для текстовых полей -->
        <Style TargetType="TextBox">
            <Setter Property="FontSize" Value="12"/>
            <Setter Property="Padding" Value="5"/>
            <Setter Property="BorderBrush" Value="#BDC3C7"/>
            <Setter Property="BorderThickness" Value="1"/>
            <Setter Property="Background" Value="White"/>
            <Style.Triggers>
                <Trigger Property="IsFocused" Value="True">
                    <Setter Property="BorderBrush" Value="#3498DB"/>
                </Trigger>
                <Trigger Property="IsMouseOver" Value="True">
                    <Setter Property="BorderBrush" Value="#7F8C8D"/>
                </Trigger>
            </Style.Triggers>
        </Style>

        <!-- Стили для ComboBox -->
        <Style TargetType="ComboBox">
            <Setter Property="FontSize" Value="12"/>
            <Setter Property="Padding" Value="3"/>
            <Setter Property="BorderBrush" Value="#BDC3C7"/>
            <Setter Property="BorderThickness" Value="1"/>
        </Style>

        <!-- Стили для DataGrid -->
        <Style TargetType="DataGrid">
            <Setter Property="FontSize" Value="11"/>
            <Setter Property="BorderBrush" Value="#34495E"/>
            <Setter Property="BorderThickness" Value="1"/>
            <Setter Property="RowDetailsVisibilityMode" Value="VisibleWhenSelected"/>
        </Style>

        <!-- Стиль заголовков DataGrid -->
        <Style x:Key="DataGridHeaderStyle" TargetType="DataGridColumnHeader">
            <Setter Property="Background" Value="#ECF0F1"/>
            <Setter Property="Foreground" Value="#2C3E50"/>
            <Setter Property="FontWeight" Value="Bold"/>
            <Setter Property="FontSize" Value="11"/>
            <Setter Property="Height" Value="30"/>
            <Setter Property="BorderBrush" Value="#BDC3C7"/>
            <Setter Property="BorderThickness" Value="0,0,1,0"/>
            <Setter Property="HorizontalContentAlignment" Value="Center"/>
        </Style>
```

Рисунок 5.5 – Глобальные стили приложения

В главном окне реализована обработка следующих событий: автоматическое заполнение списков при старте программы, отображение данных о продажах при выделении конкретного партнера, а также открытие вспомогательных форм при активации соответствующих команд в меню. Примеры обработчиков приведены на рисунке 5.6.

```
Ссылок: 9
private void LoadData()
{
    try
    {
        _context = new AppDbContext();

        var partners = _context.Partners
            .Include(p => p.PartnerType)
            .Include(p => p.Sales)
                .ThenInclude(s => s.Product)
            .ToList();

        _partners = partners.Select(p => new PartnerViewModel(p)).ToList();
        PartnersListBox.ItemsSource = _partners;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка загрузки данных: {ex.Message}",
                        "Ошибка", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
```

Рисунок 5.6 – Метод загрузки данных

В диалоговых окнах выполняется проверка корректности вводимых данных (рисунок 5.7) и обрабатываются ошибки, возникающие при сохранении.

```
Ссылок: 1
private void SaveButton_Click(object sender, RoutedEventArgs e)
{
    try
    {
        if (string.IsNullOrWhiteSpace(NameTextBox.Text))
        {
            MessageBox.Show("Введите наименование партнера", "Предупреждение",
                            MessageBoxButton.OK, MessageBoxImage.Warning);
            NameTextBox.Focus();
            return;
        }

        if (TypeComboBox.SelectedItem == null)
        {
            MessageBox.Show("Выберите тип партнера", "Предупреждение",
                            MessageBoxButton.OK, MessageBoxImage.Warning);
            TypeComboBox.Focus();
            return;
        }

        // Заполняем все поля
        _partner.Name = NameTextBox.Text.Trim();
        _partner.TypeId = (int)TypeComboBox.SelectedValue;

        // ДОБАВЛЯЕМ СОХРАНЕНИЕ АДРЕСА
        _partner.Address = string.IsNullOrWhiteSpace(AddressTextBox.Text)
            ? null
            : AddressTextBox.Text.Trim();

        if (!string.IsNullOrWhiteSpace(RatingTextBox.Text))
        {
            if (int.TryParse(RatingTextBox.Text, out int rating) && rating >= 0)
            {
                _partner.Rating = rating;
            }
            else
            {
                MessageBox.Show("Рейтинг должен быть целым неотрицательным числом",
                                "Предупреждение", MessageBoxButton.OK, MessageBoxImage.Warning);
                RatingTextBox.Focus();
                return;
            }
        }

        _partner.DirectorFullname = string.IsNullOrWhiteSpace(DirectorTextBox.Text) ? null : DirectorTextBox.Text.Trim();
        _partner.Phone = string.IsNullOrWhiteSpace(PhoneTextBox.Text) ? null : PhoneTextBox.Text.Trim();
        _partner.Email = string.IsNullOrWhiteSpace(EmailTextBox.Text) ? null : EmailTextBox.Text.Trim();
```

Рисунок 5.7 – Валидация и сохранение данных

Окончательный вариант оформления основного окна программы показан на рисунке 5.8.

Рисунок 5.8 – Главное окно приложения

Заключение

За период прохождения практики с 9 по 17 марта 2026 года были углублены теоретические знания и усовершенствованы профессиональные навыки.

Достигнутые результаты:

- На основе изучения проектной и технической документации сформулированы требования к программным модулям, обеспечивающие их согласованное функционирование в общей структуре разрабатываемого ПО.
- Выполнено моделирование этапов разработки, позволившее упорядочить процесс объединения компонентов и выстроить логику их взаимодействия.
- Проведена отладка программных компонентов с применением профильных инструментов, что дало возможность обнаружить и исправить неполадки до завершения разработки.
- Созданы тестовые комплекты и сценарии для оценки работоспособности и устойчивости программного продукта.

Приобретенные практические навыки:

- объединение отдельных модулей в целостную программную систему;
- диагностика и исправление ошибок в коде с использованием отладочных средств;
- формирование тестовых наборов и сценариев для проверки ПО;
- анализ исходного кода на предмет соблюдения установленных правил оформления.

Приложения

ПРИЛОЖЕНИЕ 1

Описание предметной области

Производственная компания «Мастер пол» выпускает напольные покрытия. Компания занимается производством и реализует свою продукцию через партнеров, которые доставляют продукцию компании до конечных потребителей.

С целью оптимизации деятельности компании разрабатывается система, в которой заказчик выделил несколько подсистем:

- продукция;
- склад и материалы;
- производство;
- сотрудники;
- партнеры.

В данной области определены следующие основные составляющие:

1. Партнеры: тип, наименование компании, юридический адрес, ИНН, ФИО директора, контактные данные (телефон, email), логотип, рейтинг, места продаж, история реализации продукции для последующего расчета скидок.

Партнеры покупают продукцию со скидками, которые зависят от общего объема их продаж. Скидки мотивируют партнеров к большему объему продаж. Партнеры реализуют товары в розничных и оптовых магазинах, интернет-магазинах, другим компаниям по продаже товаров и оказанию услуг.

Взаиморасчеты производственной компании с партнерами осуществляются в рублях.

2. Менеджеры: поиск и регистрация партнеров, решение об изменении рейтинга партнера, ведение истории изменений рейтинга, прием

заявок от партнеров, формирование для них предложения на основании статистики истории продаж, полученной от аналитиков компании.

Продолжение приложения 1

3. Заявка: если партнер ранее производил закупку продукции и предоставил данные о ее реализации, то на основании этих данных и текущих объемах готовой продукции менеджер формирует предложение. Если работа ведется с новым партнером, то менеджер предоставляет каталог готовой продукции и принимает заявку. Затем менеджер должен указать стоимость и дату производства каждой указанной в заявке единицы продукции. После согласования списка продукции с количеством, стоимостью и сроком изготовления заявка считается созданной.

Партнер может отменить заявку до момента внесения предоплаты. Если предоплата не поступает в течение трех дней, то заявка отменяется автоматически, о чем обязательно уведомляется партнер. После поступления предоплаты продукция, указанная в заявке и отсутствующая на складе в нужном количестве, запускается в производство. По готовности всей продукции менеджер должен предложить организовать доставку. В момент получения продукции (путем доставки или самовывоза) партнер производит полную оплату. После проверки количества и качества продукции партнером заявка считается выполненной.

4. Сотрудники: ФИО, дата рождения, паспортные данные, банковские реквизиты, наличие семьи, состояние здоровья.

5. Кадры: допуск к работе с конкретным оборудованием. Мастер производства занимается технологией производства, контролем качества готовой продукции, заказом материалов у поставщиков.

6. Доступ: двери открываются при использовании карты сотрудника, система турникетов передает данные о перемещении сотрудников в приложение для формирования аналитиками статистики по сотрудникам.

7. Материалы (сырье): тип, наименование, поставщик, количество в

упаковке, единица измерения, описание, изображение, стоимость, количество на складе, минимальное допустимое количество, история изменений количества

Продолжение приложения 1

материалов на складе.

8. Склад: регистрация поступлений материалов, резервирование материалов для производства, отпуск и списание материалов со склада в производство, поступление и выбытие готовой продукции, получение информации о текущих остатках и движении товарно-материальных ценностей по складу.

9. Поставщики: тип, наименование, ИНН, история поставок материалов. Менеджеры контролируют список поставщиков, аналитики формируют информацию о качестве поставляемых материалов для принятия решения о прекращении работы с определенными компаниями.

10. Продукция: артикул, тип, наименование, описание, изображение, минимальная стоимость для партнера, размер упаковки (длина, ширина, высота), вес без упаковки, вес с упаковкой, сертификат качества (скан документа), номер стандарта, история изменений минимальной стоимости для партнера, время изготовления, себестоимость, номер цеха, количество человек на производстве, необходимые материалы для производства.